# Первоначальная настройка git

**Лабораторная работа № 3 — отчёт по лабораторной работе № 2 в формате Markdown (зачётная книжка № 1132259051), РУДН**

Чжан Вэньцзюнь

2026-10-20

# Содержание

**1 Цель работы** **5**

**2 Задание** **6**

**3 Теоретическое введение** **7**

**4 Выполнение лабораторной работы** **8**
4.1 Установка программного обеспечения . . . . . 8
4.2 Базовая настройка git . . . . . 9
4.3 Создание ключей SSH . . . . . 10
4.4 Создание ключа PGP . . . . . 10
4.5 Настройка подписи коммитов . . . . . 11
4.6 Создание рабочего каталога и клонирование репозитория . . . . . 12
4.7 Проверка подписи коммитов . . . . . 13

**5 Контрольные вопросы** **14**

**6 Выводы** **16**

**Список литературы** **17**

# Список иллюстраций

4.1 Установка git и gh . . . . . 8
4.2 Глобальная конфигурация git . . . . . 9
4.3 Создание ключей SSH . . . . . 10
4.4 Ключ PGP и итоговая конфигурация git . . . . . 11
4.5 Клонирование репозитория курса . . . . . 12
4.6 Проверка подписи коммита . . . . . 13

# Список таблиц

# 1 Цель работы

1. Изучить идеологию и применение средств контроля версий.
2. Освоить умения по работе с git.
3. Настроить базовую конфигурацию git, создать ключи SSH и PGP, настроить подпись коммитов.
4. Создать репозиторий курса и настроить рабочее пространство.

# 2 Задание

1. Создать базовую конфигурацию для работы с git.
2. Создать ключ SSH.
3. Создать ключ PGP.
4. Настроить подписи git.
5. Зарегистрироваться на GitHub и создать репозиторий курса.
6. Создать локальный каталог для выполнения заданий по предмету.

# 3 Теоретическое введение

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. При внесении изменений в содержание проекта система позволяет их фиксировать, совмещать изменения разных участников и выполнять откат к любой более ранней версии.

В классических (централизованных) системах используется единый репозиторий, доступ к которому предоставляется через сервер. В децентрализованных системах, к которым относится git, каждый участник имеет полную копию репозитория вместе со всей историей изменений, что позволяет работать автономно и синхронизироваться по мере необходимости [1].

Основные понятия git: хранилище (репозиторий), коммит (фиксация состояния), история (последовательность коммитов), рабочая копия (файлы в текущем каталоге). Работа с командами оболочки и перенаправлением вывода подробно рассмотрена в [2].

# 4 Выполнение лабораторной работы

## 4.1 Установка программного обеспечения

Установлены пакеты `git` и `gh` (интерфейс командной строки GitHub):

```
sudo dnf -y install git gh
```

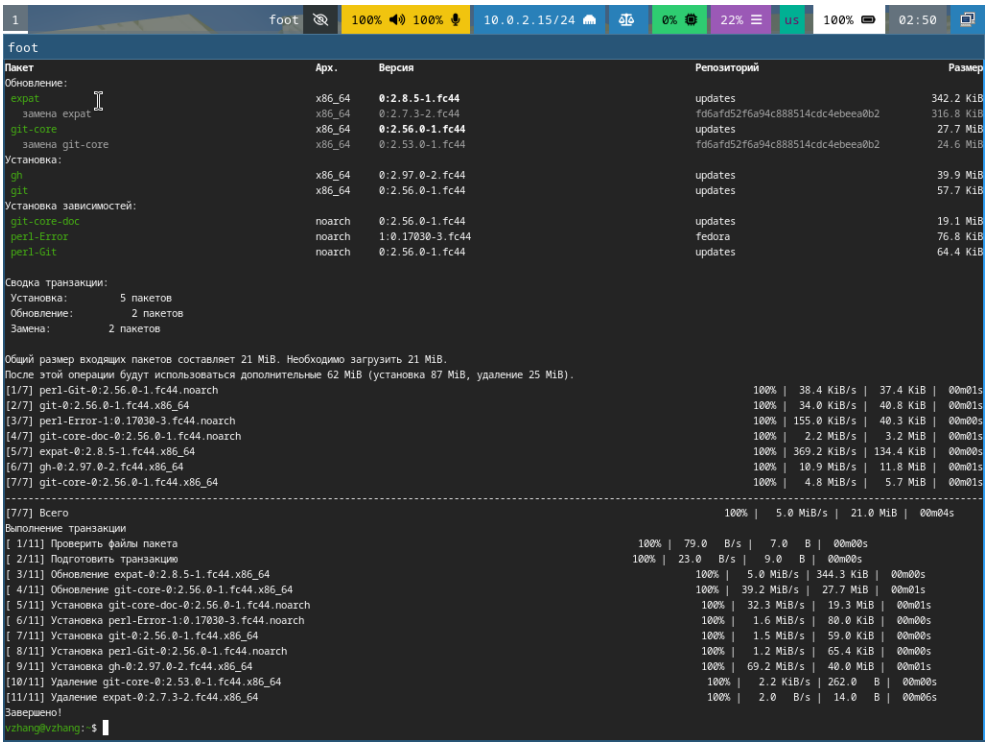



Рисунок 4.1: Установка git и gh

## 4.2 Базовая настройка git

Заданы глобальные параметры: имя пользователя, адрес электронной почты, поддержка UTF-8 в именах файлов, имя начальной ветки, настройки обработки переносов строк:

```
git config --global user.name 'Чжан Вэньцзюнь'
git config --global user.email wenjun-code@users.noreply.github.com
git config --global core.quotepath false
git config --global init.defaultBranch master
git config --global core.autocrlf input
git config --global core.safecrlf warn
git config --global --list
```

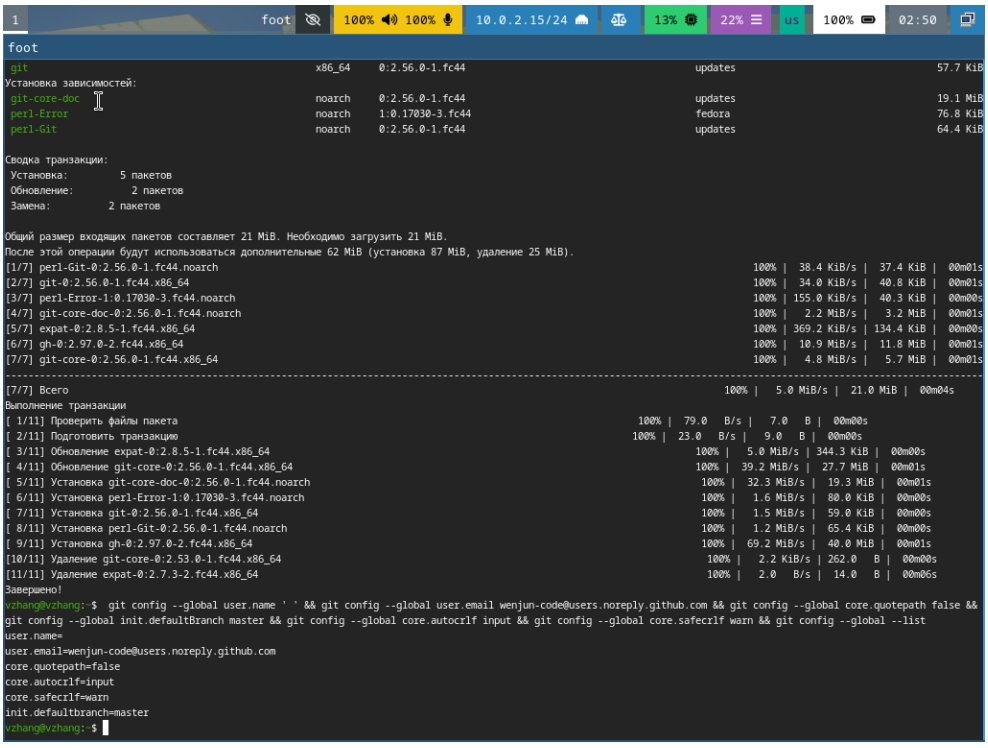



Рисунок 4.2: Глобальная конфигурация git

## 4.3 Создание ключей SSH

Сгенерированы два ключа: RSA длиной 4096 бит и Ed25519. Параметр `-N ''` задаёт пустую фразу-пароль, параметр `-q` отключает вывод случайного рисунка:

```
ssh-keygen -t rsa -b 4096 -C wenjun-code@users.noreply.github.com -f ~/.ssh/id_rsa -N
ssh-keygen -t ed25519 -C wenjun-code@users.noreply.github.com -f ~/.ssh/id_ed25519 -N
ls -l ~/.ssh
```

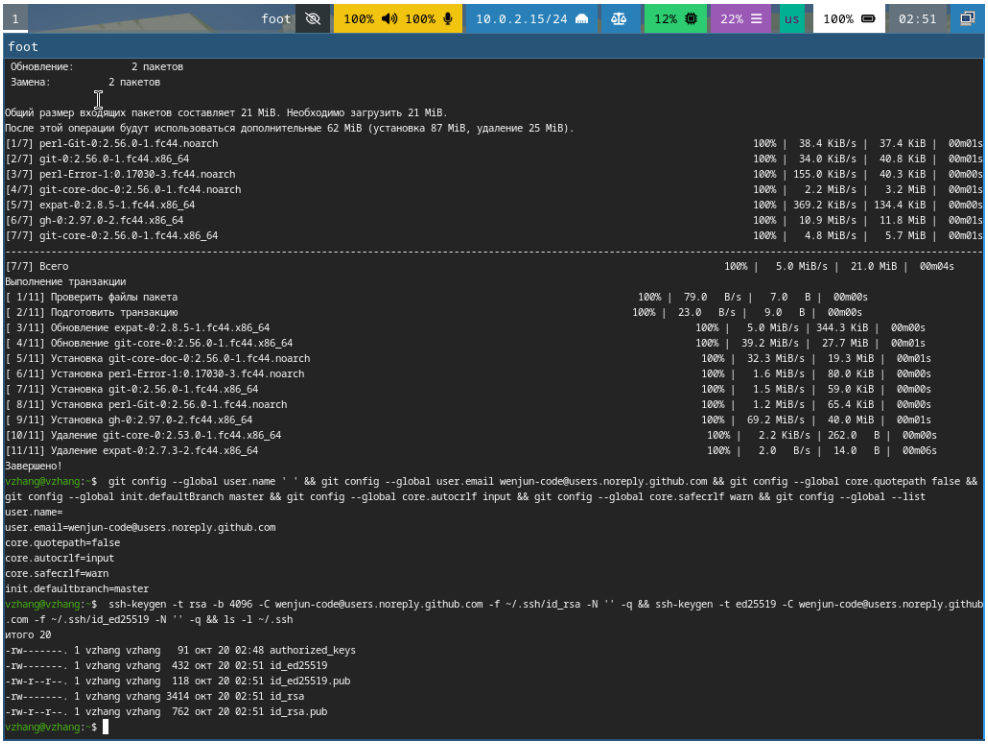



Рисунок 4.3: Создание ключей SSH

Ключи размещены в каталоге `~/.ssh/`: закрытые `id_rsa` и `id_ed25519`, открытые `id_rsa.pub` и `id_ed25519.pub`.

## 4.4 Создание ключа PGP

Ключ создан в пакетном режиме с алгоритмом RSA 4096 бит, назначением `sign` и бессрочным сроком действия:

```
gpg --batch --pinentry-mode loopback --passphrase '' \
    --quick-gen-key 'Чжан Вэньцзюнь <wenjun-code@users.noreply.github.com>' rsa4096 s
gpg --list-secret-keys --keyid-format LONG
```

Отпечаток созданного ключа: `24299B5D6A1430222C7F881072F4C76DCD8F1C14`, идентификатор `72F4C76DCD8F1C14`.

## 4.5 Настройка подписи коммитов

Указан ключ подписи, включена автоматическая подпись всех коммитов:

```
git config --global user.signingkey 24299B5D6A1430222C7F881072F4C76DCD8F1C14
git config --global commit.gpgsign true
git config --global gpg.program /usr/bin/gpg
```

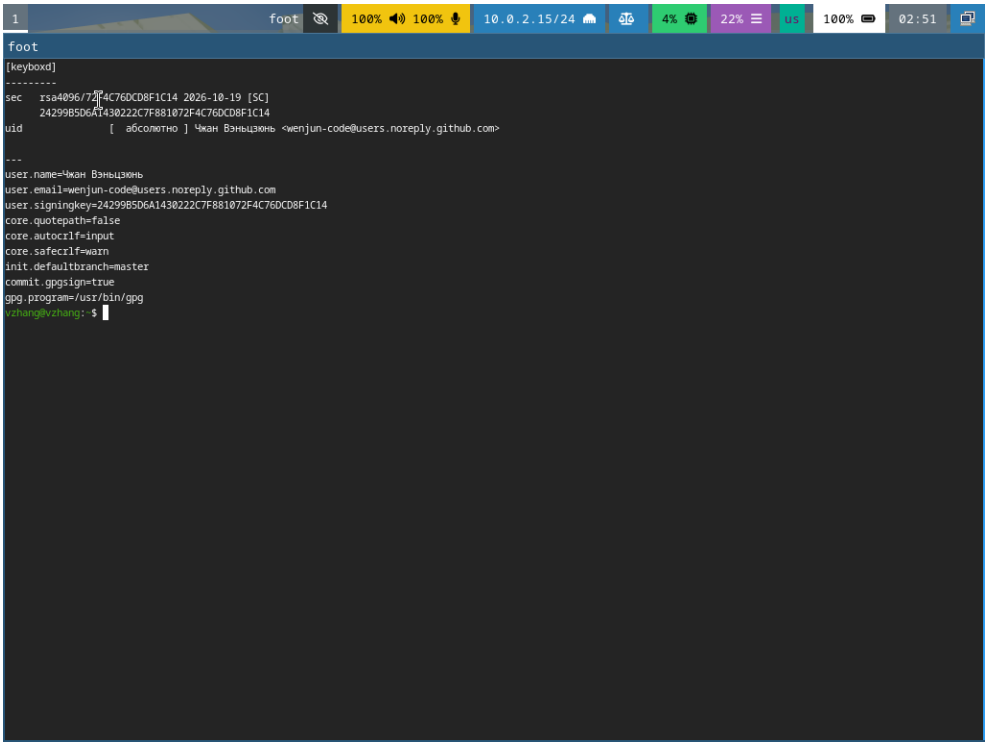



Рисунок 4.4: Ключ PGP и итоговая конфигурация git

Открытая часть ключа экспортирована для добавления в учётную запись GitHub:

```
gpg --armor --export 24299B5D6A1430222C7F881072F4C76DCD8F1C14 > gpg-public.asc
```

## 4.6 Создание рабочего каталога и клонирование репозитория

Создан каталог для выполнения заданий по предмету, в него склонирован репозиторий курса:

```
mkdir -p ~/work/study/2026-2027
cd ~/work/study/2026-2027
git clone https://github.com/wenjun-code/linux_2026-2027
ls -la
```

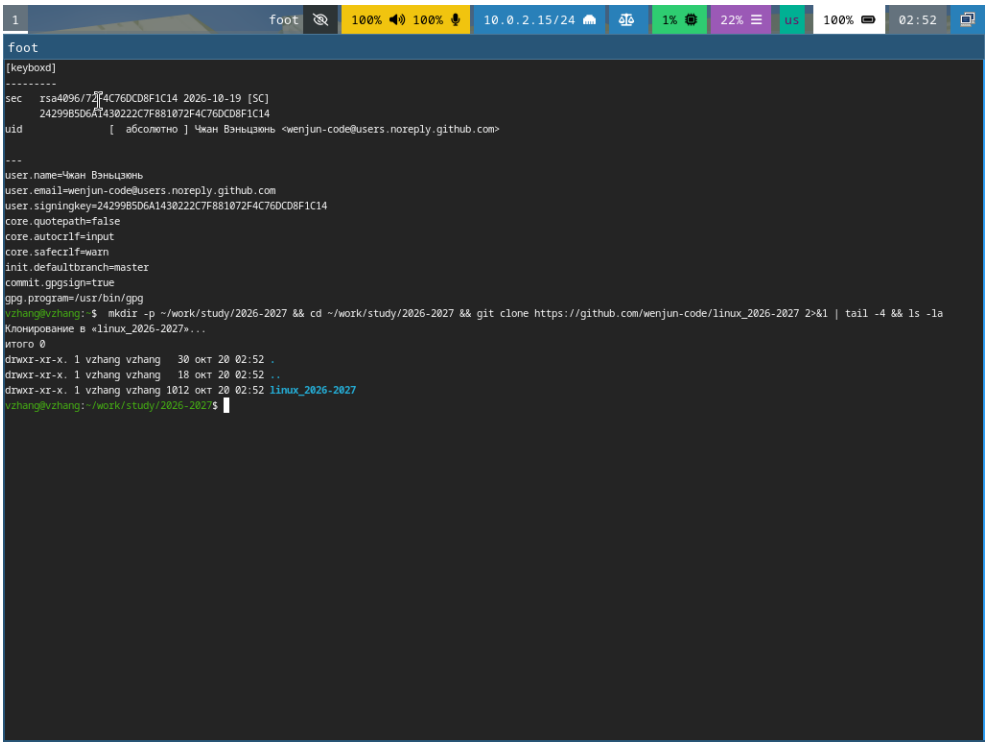



Рисунок 4.5: Клонирование репозитория курса

## 4.7 Проверка подписи коммитов

Выполнен пробный коммит с явным указанием ключа подписи (`-S`) и проверка подписи:

```
git add sign-test.txt
git commit -S -m 'test: signed commit'
git log --show-signature -1
```

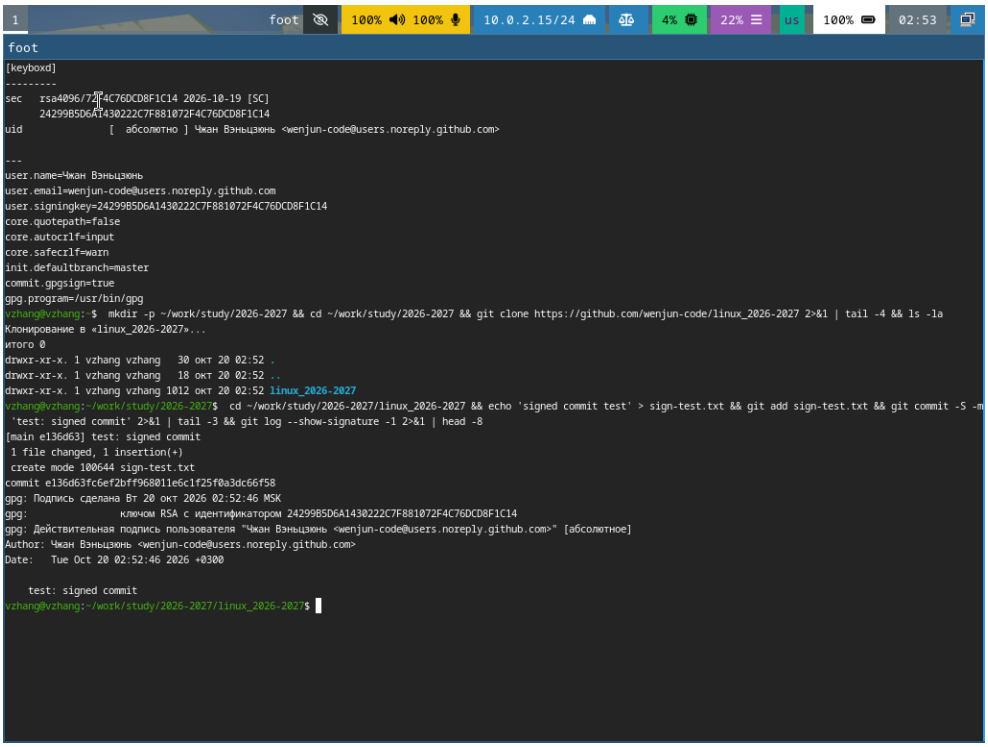



Рисунок 4.6: Проверка подписи коммита

Утилита gpg подтвердила корректность подписи: `gpg: Действительная подпись пользователя "Чжан Вэньцзюнь <wenjun-code@users.noreply.github.com>"`. На стороне GitHub такой коммит отображается с меткой `Verified`.

# 5 Контрольные вопросы

**1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?**

VCS — программное обеспечение для управления изменением версий документов и кода. Решает задачи хранения истории изменений, совместной работы, отката к предыдущим версиям, ветвления разработки и резервного копирования.

**2. Объясните понятия VCS и их отношения: хранилище, commit, история, рабочая копия.**

Хранилище (репозиторий) — база данных версий проекта. Commit — фиксация состояния файлов, сохраняемая в хранилище вместе с автором, датой и сообщением. История — последовательность коммитов, связанных отношениями «родитель — потомок». Рабочая копия — файлы проекта в текущем каталоге, из которых вносимые изменения попадают в коммит.

**3. Чем отличаются централизованные и децентрализованные VCS?**

В централизованных (SVN, CVS) существует единое хранилище на сервере, а у пользователей — только рабочая копия. В децентрализованных (git, Mercurial) каждый пользователь получает полную копию хранилища с историей, поэтому может работать автономно.

**4. Действия с VCS при единоличной работе с хранилищем.**

Инициализация хранилища, создание файлов, добавление изменений в индекс, фиксация коммитов, просмотр истории, при необходимости — откат к предыдущей версии.

**5. Порядок работы с общим хранилищем.**

Получение актуальной версии (`git pull`), создание ветки под задачу, внесение изменений, фиксация коммитов, отправка в удалённое хранилище (`git push`), создание запроса на слияние и разрешение конфликтов.

**6. Каковы основные задачи, решаемые git?**

Управление версиями, ветвление и слияние, распределённая работа, хранение истории, тегирование выпусков.

**7. Назовите и дайте характеристику командам git.**

`init` — создание хранилища; `clone` — копирование удалённого хранилища; `add` — добавление в индекс; `commit` — фиксация; `push` и `pull` — обмен с удалённым хранилищем; `branch` и `checkout` — работа с ветками; `merge` — слияние; `log` — история; `status` — состояние рабочей копии.

**8. Примеры работы с локальным и удалённым репозиториями.**

Локально: `git init`, `git add .`, `git commit -m 'первый коммит'`. С удалённым: `git remote add origin <url>`, `git push -u origin master`, `git pull`.

**9. Что такое и зачем нужны ветви (branches)?**

Ветвь — независимая линия разработки, ответвлённая от основной истории. Ветви позволяют разрабатывать функции и исправления изолированно, не нарушая стабильность основной ветки, и затем сливать результаты.

**10. Как и зачем можно игнорировать некоторые файлы при commit?**

С помощью файла `.gitignore`, в котором перечисляются шаблоны имён файлов и каталогов. Игнорируются временные файлы редакторов, объектные файлы сборки, зависимости, файлы с секретами и локальные конфигурации.

# 6 Выводы

В ходе работы изучены идеология и применение средств контроля версий, освоены базовые приёмы работы с git. Создана глобальная конфигурация git, сгенерированы ключи SSH двух типов и ключ PGP, настроена автоматическая подпись коммитов. Подпись коммита проверена утилитой `gpg` и признана действительной. Репозиторий курса склонирован в рабочий каталог. Все задачи лабораторной работы выполнены.

# Список литературы

1. Таненбаум Э., Бос Х. Современные операционные системы. 4-е изд. СПб.: Питер, 2015. 1120 с.
2. Robbins A. Bash Pocket Reference. O'Reilly Media, 2016. 156 с.